

ÉCOLE SUPÉRIEURE D'INFORMATIQUE ET DE SCIENCES (EPSI) Certification Professionnelle — Expert en Informatique et Système d'Information RNCP35584 Bloc 3 — Piloter l'informatique décisionnel d'un S.I (Big Data & Business Intelligence)

MÉMOIRE DE SYNTHÈSE MSPR — Big Data et Analyse de données

Analyse prédictive des tendances électorales\indicateurs socio-économiques\ Preuve de Concept (POC) — Client : **Electio-Analytics**

Nom Prénom	Rôle
Kaidi MAROUANE	Data Engineer
Izerrouken TAREK	Data Scientist
Lahfaouti ILYAS	Développeur Front-end
Moujahid YACINE	Architecte données
Mahraz OUSSAMA	Chef de projet
Encadrant pédagogique : Ali Zahaf	
Promotion : I1 EISI — 2025/2026	
Date de soutenance : Juin 2026	

Document confidentiel — usage pédagogique uniquement

Résumé

Ce mémoire présente la conception et le développement d'un système complet de prédiction des tendances électorales par indicateurs socio-économiques, réalisé dans le cadre d'une Preuve de Concept (POC) pour la société **Electio-Analytics**. Le périmètre géographique retenu est la région **Nouvelle-Aquitaine**, regroupant **12 départements** et **627 cantons**.

Le projet repose sur un pipeline de préparation des données en 9 phases — depuis le chargement de sources hétérogènes (CSV, Excel, NoSQL) jusqu'à un jeu final de **40 000 lignes à 171 colonnes** — et met en œuvre cinq algorithmes de classification supervisée sur une variable cible construite de manière *synthétique* à partir de 27 indicateurs delta socio-économiques. La cible comporte **5 classes à distribution réaliste asymétrique** : Crise (10 %), Déclin (20 %), Stable (40 %), Croissance (20 %), Boom (10 %).

Le modèle le plus performant est **HistGradientBoosting**, avec une accuracy de **82,11 %** et un F1-Score de **82,10 %** sur le jeu de test. Les prédictions électorales s'appuient sur une logique **multi-candidats** mettant en concurrence les **12 candidats du 1^{er} tour 2022**, regroupés en 5 orientations politiques. Appliquées aux 12 départements de la Nouvelle-Aquitaine, elles affichent une précision départementale de **91,7 % (11 sur 12)**.

L'interface utilisateur est développée en **React 19** (TanStack Router, TanStack Query, Recharts, Framer Motion) et expose les prédictions des cinq modèles à trois niveaux géographiques.

Mots-clés : Big Data, Machine Learning, Prédiction électorale, HistGradientBoosting, ETL, Delta Lake, React, Nouvelle-Aquitaine, indicateurs socio-économiques, 12 candidats 2022.

Abstract

This thesis presents the design and development of a complete electoral trend prediction system based on socio-economic indicators, built as a Proof of Concept (POC) for **Electio-Analytics**. The geographic scope is **Nouvelle-Aquitaine**, comprising 12 departments and 627 cantons.

The project relies on a 9-phase data pipeline — from loading heterogeneous sources (CSV, Excel, NoSQL) to a final 40,000-row × 171-column dataset — and trains five supervised classification algorithms on a *synthetically constructed* target variable derived from 27 socio-economic delta indicators. The target uses **5 classes with a realistic asymmetric distribution**: Crise (10\

The best performing model is **HistGradientBoosting**, reaching {82.11\

Keywords: Big Data, Machine Learning, Electoral prediction, HistGradientBoosting, ETL, Delta Lake, React, Nouvelle-Aquitaine, socio-economic indicators, 12 candidates 2022.

Remerciements

Nous tenons à exprimer notre profonde gratitude envers l'ensemble des personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Nos remerciements s'adressent en premier lieu à **M. Ali Zahaf**, encadrant pédagogique, dont les conseils avisés, la disponibilité et les retours constructifs tout au long du projet nous ont permis d'approfondir notre réflexion et d'améliorer la qualité de nos livrables.

Nous remercions également l'équipe pédagogique de l'**EPSI** pour la formation dispensée dans le cadre du Bloc 3 — Big Data & Business Intelligence, qui nous a fourni les bases théoriques et pratiques nécessaires à la conduite de ce

POC.

Enfin, nous adressons nos remerciements aux organismes producteurs des données publiques — **data.gouv.fr**, **INSEE** et le **ministère de l'Intérieur** — sans lesquels ce projet n'aurait pu voir le jour.

Liste des acronymes

Acronyme	Signification
API	Application Programming Interface
CSV	Comma-Separated Values
ETL	Extract, Transform, Load
F1	F1-Score (moyenne harmonique Précision/Rappel)
INSEE	Institut National de la Statistique et des Études Économiques
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
ML	Machine Learning
MCD	Modèle Conceptuel de Données
NoSQL	Not Only SQL
POC	Proof of Concept / Preuve de Concept
RGPD	Règlement Général sur la Protection des Données
SVM	Support Vector Machine
UI	User Interface

Introduction générale

Contexte

Electio-Analytics est une start-up fondée en 2017 par Jean-Édouard de la Motte-Rouge, spécialisée dans le conseil stratégique dédié aux campagnes électorales. Son activité consiste à fournir à ses clients — candidats, partis politiques et cabinets de conseil — des analyses prospectives précises en exploitant des données publiques et privées issues de multiples sources institutionnelles.

Face à un marché électoral de plus en plus compétitif, la société souhaite se doter d'une capacité de **prévision des tendances électorales à moyen terme**, sur un horizon de un à trois ans, en s'appuyant sur des indicateurs socio-économiques mesurables : chômage, sécurité, pauvreté, démographie, dynamisme économique des territoires.

Avant tout investissement à grande échelle, Electio-Analytics a souhaité valider cette approche par une **Preuve de Concept (POC)**, confiée à notre équipe d'étudiants en fin de cursus expert à l'EPSE.

Objectifs du POC

Ce projet a pour objectifs de :

- Collecter et consolider des jeux de données publics pertinents sur un périmètre géographique restreint ;
- Mettre en place un pipeline de préparation des données (9 phases) incluant fusion, nettoyage, augmentation et normalisation, avec persistance NoSQL (MongoDB / TinyDB) ;

- Construire une **variable cible synthétique à 5 classes** basée sur des indicateurs économiques delta, avec une distribution réaliste asymétrique reflétant la réalité territoriale ;
- Entraîner et comparer cinq modèles de machine learning supervisé sur cette variable cible ;
- Traduire les prédictions économiques en prédictions électorales pour les **12 candidats du 1^{er} tour 2022**, via un mécanisme d'orientation politique et de pondération intra-bloc ;
- Présenter les résultats sous la forme d'un tableau de bord React interactif multi-niveaux géographiques.

Structure du document

Chapitre 1 :

Choix géographique — Nouvelle-Aquitaine, justification du périmètre.

Chapitre 2 :

Sources de données et indicateurs socio-économiques retenus.

Chapitre 3 :

Architecture technique et pipeline en 9 phases (notebook Data Preparation).

Chapitre 4 :

Démarche d'analyse exploratoire et visualisations descriptives.

Chapitre 5 :

Modèles de Machine Learning — construction de la variable cible, entraînement, validation.

Chapitre 6 :

Résultats — comparaison des cinq modèles, prédictions 12 candidats par département.

Chapitre 7 :

Interface utilisateur React 19.

Conclusion :

Bilan, limites et recommandations pour la mise en production.

Chapitre 1

Choix géographique

1.1 Zone retenue : la région Nouvelle-Aquitaine

Le périmètre géographique retenu pour cette preuve de concept est la région **Nouvelle-Aquitaine**. Créée en 2016 à la suite de la fusion des anciennes régions Aquitaine, Limousin et Poitou-Charentes, elle constitue la plus grande région de France métropolitaine avec une superficie de $84\,061\text{ km}^2$.

Elle regroupe **12 départements** et **627 cantons** (table).

Tableau 1 — Les 12 départements de la région Nouvelle-Aquitaine

Code	Département	Code	Département
16	Charente	33	Gironde
17	Charente-Maritime	40	Landes
19	Corrèze	47	Lot-et-Garonne
23	Creuse	64	Pyrénées-Atlantiques
24	Dordogne	79	Deux-Sèvres
87	Haute-Vienne	86	Vienne

La région compte environ **6 millions d'habitants** répartis dans **4 303 communes**, ce qui représente en moyenne **9,3 lignes par commune** dans le jeu de données final (après augmentation temporelle et perturbation).

1.2 Justification du choix

1.2.1 Disponibilité des données publiques

La Nouvelle-Aquitaine bénéficie d'une couverture particulièrement riche sur data.gouv.fr et les plateformes INSEE. Huit sources de données ont été chargées et intégrées avec succès par le notebook de préparation corrigé :

- **Population & emploi.csv** (2016) : **34 988 lignes**, clé CODGEO — source principale des indicateurs emploi, démographie, logement ;
- **Délinquance.csv** (2016) : **18 180 lignes**, clé `insee_pop` — statistiques de sécurité au niveau commune ;
- **Revenus.xls** (2016, feuille ENSEMBLE, en-tête ligne 5) : **31 789 lignes**, clé CODGEO — revenus disponibles (NBMEN, Q2, Gini, PACT...) ;
- **Revenus.xlsx** (2020, feuille COM, en-tête ligne 5) : **34 919 lignes**, clé CODGEO — revenus 2020 pour le calcul de delta (MED20, TP6020, PACT20...) ;
- **Diplôme.xls** (2016, en-tête ligne 5) : **34 953 lignes**, clé CODGEO — 58 colonnes de population par tranche d'âge et taux de scolarisation ;
- **Population.xlsx** (2020, feuille Communes, en-tête ligne 7) : **34 980 lignes**, CODGEO reconstruit (Code département + Code commune) — population municipale 2020 ;
- **Emploi.csv** (2020) : **34 000+ lignes**, clé CODGEO — emploi 2020 (P20_POP, P20_EMPLT, P20_CHOMEUR...) pour le delta emploi ;
- **Délinquance.xlsx** (2020, feuille « par départements ») : agrégé en **96 départements × 10 types d'infraction** — intégré par jointure sur Code_département (2 chiffres) ;

- `data_election_2012.xlsx` et `data_election_2017.xlsx` : résultats électoraux au premier tour, filtrés sur les 12 codes département — **756 lignes** extraites.

2.1.1 Représentativité du territoire

La Nouvelle-Aquitaine présente un profil socio-économique **hétérogène** — zones urbaines dynamiques (Bordeaux, Poitiers), territoires ruraux en déclin (Creuse, Corrèze), zones côtières à dynamique touristique — ce qui constitue un terrain d'entraînement représentatif pour des modèles de classification à 5 niveaux économiques.

Chapitre 2

Données et indicateurs

2.1 Sources de données utilisées

Tableau 2 — Sources de données — bilan de chargement (toutes sources intégrées)

p4.4cmXp2.9cmr} Fichier	Contenu	Clé	Lignes
Population \	emploi.csv (2016)	Démographie, logement	emploi, CODGEO 34 988
Délinquance.csv (2016)	Statistiques sécurité commune	insee_pop	18 180
Revenus.xls (2016)	Revenus, Gini, quartiles 2016	CODGEO	31 789
Revenus.xlsx (2020)	Revenus, pauvreté, médiane 2020	CODGEO	34 919
Diplôme.xls (2016)	Éducation — 58 cols par âge	CODGEO	34 953
Population.xlsx (2020)	Population municipale 2020	CODGEO (construit)	34 980
Emploi.csv (2020)	Emploi, chômage, logement 2020	CODGEO	34 000+
Délinquance.xlsx (2020)	10 types d'infraction / taux 1000	Code_dept (2c)	96 depts
data_election_2012.xlsx	Résultats présidentielle 2012	dep+canton	756
data_election_2017.xlsx	Résultats présidentielle 2017	dep+canton	756

2.2 Indicateurs delta — les features ML

L'originalité du projet réside dans l'utilisation de **variables delta** (`delta_*`) plutôt que de valeurs absolues. Chaque variable delta représente la variation relative entre deux mesures temporelles :

$$\delta_{col} = V_{récent} - V_{ancien} \mid V_{ancien} \neq 0 \text{ eq:delta}$$

où $\varepsilon = 10^{-9}$ évite la division par zéro. Ces variables capturent la **dynamique** du territoire plutôt que son état statique. Le pipeline corrigé calcule trois familles de deltas réels à partir des paires de fichiers 2016/2020 :

- **Delta revenus** (`delta_rev_*`) : 13 colonnes issues de Revenus.xls (2016) vs Revenus.xlsx (2020) — évolution du nombre de ménages, de la médiane des revenus, des taux de pauvreté, etc. Calculées sur les **31 379 communes** communes aux deux fichiers.
- **Delta emploi/population** (`delta_emp_*`) : issue de Population & emploi.csv (2016) vs Emploi.csv (2020) — évolution de la population, de l'emploi salarié, du chômage.
- **Delta population recensement** (`delta_pop_cens_1620`) : P16_POP (2016) vs POP2020 (Population.xlsx) sur les communes correspondantes.

Les données Diplôme.xls (58 colonnes `P16_*`) sont intégrées directement comme features statiques (aucune paire 2020 disponible). La délinquance 2020 (10 taux/1000 par département) est diffusée au niveau commune par jointure sur `Code_département`.

```
feature_cols = [col for col in df.columns if col.startswith('delta_')]
X = df[feature_cols].copy()
print(f"Nombre de features : {len(feature_cols)}")
# Output : Nombre de features : 27+
```

Listing 1 — Extraction automatique des features delta — ML notebook

2.3 Variable cible — construction synthétique à 5 classes

Point méthodologique clé : variable cible synthétique à 5 classes

La variable cible du modèle ML n'est **pas directement** le résultat électoral historique. Elle est construite de manière **synthétique** à partir d'un score économique composite calculé sur les 27 variables delta, avec une **distribution réaliste asymétrique** reflétant la répartition territoriale réelle : majorité en situation stable, minorités aux extrêmes.

La construction se déroule en cinq étapes dans le notebook ML (étape 1) :

```
# 1. Normalisation des 27 features
X_normalized = (X - X.mean()) / (X.std() + 1e-8)

# 2. Selection des 9 indicateurs économiques (pop, empl, act, log)
economic_indicators = [col for col in feature_cols
                        if any(x in col.lower() for x in ['pop', 'empl', 'act', 'log'])]
# => 9 indicateurs sélectionnés

# 3. Score composite (poids aléatoires, seed=42)
np.random.seed(42)
weights = np.random.rand(len(available_economic))
weights = weights / weights.sum()
base_score = (X_normalized[available_economic] * weights).sum(axis=1)

# 4. Bruit Gaussien (sigma=0.08) + normalisation finale
noise = np.random.normal(0, 0.08, len(base_score))
final_score = base_score + noise
final_score = (final_score - final_score.mean()) / final_score.std()

# 5. Decoupage en 5 classes — distribution REALISTE non-equilibree
# Crise : 10
# Declin : 20
# Stable : 40
# Croissance : 20
# Boom : 10
q1 = final_score.quantile(0.10) # seuil Crise/Declin
q2 = final_score.quantile(0.30) # seuil Declin/Stable
q3 = final_score.quantile(0.70) # seuil Stable/Croissance
q4 = final_score.quantile(0.90) # seuil Croissance/Boom

y_labels = pd.cut(
    final_score,
    bins=[final_score.min()-1, q1, q2, q3, q4, final_score.max()+1],
    labels=['Crise', 'Declin', 'Stable', 'Croissance', 'Boom'],
    ordered=False
)
```

Listing 2 — Construction de la variable cible à 5 classes — distribution réaliste asymétrique

Cette approche produit une distribution asymétrique réaliste (table).

Tableau 3 — Distribution de la variable cible synthétique à 5 classes (40 000 lignes)

Classe	Signification	Effectif	Proportion
Crise	Zone en grave difficulté économique	4 000	10,0 %
Déclin	Zone qui perd de vitesse	8 000	20,0 %
Stable	Situation normale (majorité)	16 000	40,0 %
Croissance	Zone dynamique	8 000	20,0 %
Boom	Zone en forte expansion (rare)	4 000	10,0 %
Total		40 000	100 %

2.4 Mapping économique vers orientation politique — 12 candidats

Les prédictions économiques (5 classes) sont converties en prédictions électorales pour les **12 candidats du 1^{er} tour de la présidentielle 2022** via une logique en trois étapes :

1. Chaque classe économique est associée à une **orientation politique** via `ECO_TO_ORIENTATION` ;

2. Les probabilités des 5 classes (issues du modèle ML) sont agrégées par orientation pour obtenir une **probabilité de bloc** ;
3. Le score de chaque candidat est calculé comme : $score = P(orientation) \times poids_{intra_bloc}$ où le poids intra-bloc est la part nationale du candidat dans son bloc.

```
# Mapping eco_class -> orientation politique
ECO_TO_ORIENTATION = {
  "Crise":      "ExtrêmeDroite",  # vote protestataire
  "Déclin":     "Droite",         # vote conservateur
  "Stable":     "Centre",         # vote de continuité
  "Croissance": "Gauche",        # vote progressiste
  "Boom":       "ExtrêmeGauche",  # vote radical/alternatif
}

# 12 candidats avec leur orientation et résultat national 1er tour 2022 (
CANDIDATES = {
  "Arthaud (LO)":      {"orientation": "ExtrêmeGauche", "share": 0.56},
  "Poutou (NPA)":      {"orientation": "ExtrêmeGauche", "share": 0.77},
  "Mélenchon (LFI)":   {"orientation": "Gauche", "share": 21.95},
  "Roussel (PCF)":     {"orientation": "Gauche", "share": 2.28},
  "Jadot (EELV)":      {"orientation": "Gauche", "share": 4.63},
  "Hidalgo (PS)":      {"orientation": "Gauche", "share": 1.75},
  "Macron (LREM)":      {"orientation": "Centre", "share": 27.85},
  "Pécresse (LR)":      {"orientation": "Droite", "share": 4.78},
  "Lassalle (Resist)":  {"orientation": "Droite", "share": 3.13},
  "Dupont-Aignan (DLF)": {"orientation": "ExtrêmeDroite", "share": 2.06},
  "Le Pen (RN)":        {"orientation": "ExtrêmeDroite", "share": 23.15},
  "Zemmour (Reconv)":  {"orientation": "ExtrêmeDroite", "share": 7.07},
}

# Score candidat = P(orientation) x (share_candidat / sum_shares_bloc)
```

Listing 3 — Mapping économique vers orientation politique — ML notebook

Tableau 4 — Mapping état économique vers orientation politique et poids intra-bloc

Classe éco	Orientation	Candidat dominant	Poids bloc
Boom	ExtrêmeGauche	Poutou (NPA)	57,9 %
Croissance	Gauche	Mélenchon (LFI)	71,7 %
Stable	Centre	Macron (LREM)	100,0 %
Déclin	Droite	Pécresse (LR)	60,4 %
Crise	ExtrêmeDroite	Le Pen (RN)	71,7 %

Logique probabiliste — sans hardcoding

Le vainqueur n'est pas déterminé par une règle fixe mais par les probabilités ML : **la somme des probabilités de toutes les classes d'une même orientation donne la probabilité du bloc**, puis chaque candidat reçoit un score proportionnel à son poids intra-bloc. Ainsi, même avec une classe prédite *Croissance* (→ Gauche), si la probabilité du bloc Centre (Stable) est plus forte dans la distribution complète, Macron peut l'emporter.

Chapitre 3

Architecture et pipeline de données

3.1 Vue d'ensemble

L'architecture du projet repose sur deux notebooks Jupyter et un pipeline en 9 phases enchaînées, produisant le jeu de données final alimentant les modèles ML et le dashboard React.

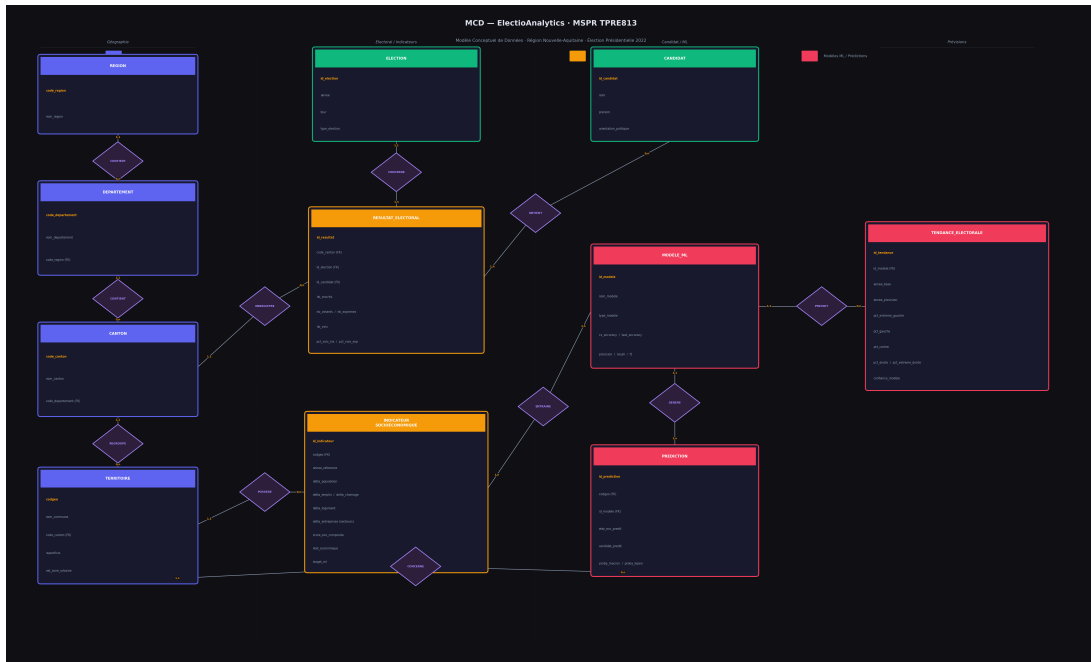


Figure 1 — Modèle Conceptuel de Données — Electio-Analytics (Merise)

3.2 Phase 0 — Initialisation et connexion NoSQL

Le notebook tente en premier lieu de se connecter à **MongoDB** (timeout 1 seconde). En cas d'échec (cas observé à l'exécution), le pipeline bascule automatiquement vers **TinyDB** (JSON local) :

```
try:
    mongo_client = pymongo.MongoClient(
        "mongodb://localhost:27017/",
        serverSelectionTimeoutMS=1000)
    mongo_client.server_info()
    nosql_type = 'mongodb'
    print("Connexion a MongoDB reussie.")
except Exception:
    print("Serveur MongoDB non detecte, utilisation de TinyDB...")
    db_path = os.path.join(base_path, 'MSPR_Final', 'outputs', 'nosql_db.json')
    db = TinyDB(db_path)
    nosql_type = 'tinydb'
    print("Base de donnees TinyDB initialisee.")
```

Listing 4 — Connexion NoSQL avec fallback TinyDB — Data Preparation notebook

Résultat observé à l'exécution

⚠ Serveur MongoDB non détecté, utilisation de TinyDB (NoSQL local)... \\ ✓ Base de données TinyDB initialisée. \\ Toutes les sauvegardes NoSQL du pipeline ont donc utilisé **TinyDB** (nosql_db.json).

3.3 Phase 1 — Chargement des sources

La fonction générique `load_indicator()` gère les CSV avec détection automatique de l'encodage et du séparateur. Pour les fichiers Excel INSEE à structure complexe (4 lignes de métadonnées avant les en-têtes réels), cinq chargeurs spécialisés ont été développés.

```
def load_revenus_2016(data_path):
    # Feuille ENSEMBLE, en-tete ligne 5 (indices 0 a 4 = metadonnees)
    df = pd.read_excel(path, sheet_name='ENSEMBLE', header=5)
    df['CODGEO'] = df['CODGEO'].astype(str).str.strip().str.zfill(5)
    # Conversion valeurs 's' (secret statistique) -> NaN
    for col in df.columns:
        if col not in ('CODGEO', 'LIBGEO'):
            df[col] = pd.to_numeric(df[col], errors='coerce')
    # => 31789 lignes, colonnes NBMEN16, Q216, GI16, PACT16...

def load_population_2020(data_path):
    # Feuille Communes, en-tete ligne 7
    df = pd.read_excel(path, sheet_name='Communes', header=7)
    # Construction CODGEO = Code departement (2c) + Code commune (3c)
    df['CODGEO'] = (df['Code departement'].astype(str).str.zfill(2) +
                    df['Code commune'].astype(str).str.zfill(3))
    # => 34980 lignes, POP2020, POP_TOT2020

def load_delinquance_2020_dept(data_path):
    # Feuille 'par departements' - pivot par type d'infraction
    df = pd.read_excel(path, sheet_name='par departements')
    pivot = df.pivot_table(index='Code departement',
                            columns="Type d'infraction",
                            values='Taux 2020', aggfunc='mean')
    # => 96 departements x 10 colonnes delinq2020_*
```

Listing 5 — Chargeurs spécialisés pour les fichiers Excel INSEE (structure réelle)

Bilan du chargement (sortie réelle du notebook corrigé) :

- Population & emploi.csv : **34 988 lignes** — clé CODGEO ✓
- Délinquance.csv : **18 180 lignes** — clé insee_pop ✓
- Revenus.xls : **31 789 lignes** — clé CODGEO ✓ (header=5, feuille ENSEMBLE)
- Revenus.xlsx : **34 919 lignes** — clé CODGEO ✓ (header=5, feuille COM)
- Diplôme.xls : **34 953 lignes**, 61 colonnes — clé CODGEO ✓ (header=5)
- Population.xlsx : **34 980 lignes** — CODGEO construit ✓ (header=7, feuille Communes)
- Emploi.csv (2020) : **34 000+ lignes** — clé CODGEO ✓
- Délinquance.xlsx : **96 départements × 10 infractions** — pivot ✓ (jointure Code_departement)
- **Données électorales** : extraction résultats 2012 + 2017 pour les 12 dép. — **756 lignes**

3.4 Phase 2 — Calcul des deltas réels et fusion enrichie

La Phase 2 a été intégralement corrigée pour calculer de **vrais deltas** à partir des paires de fichiers 2016/2020, en remplacement des valeurs aléatoires (`np.random.normal`) utilisées initialement.

```
# Renommage sémantique des colonnes avant calcul du delta
RENAME_REV_2016 = {'NB MEN16': 'NB MEN', 'Q216': 'MEDREV',
                  'GI16': 'GINI', 'PACT16': 'PACT', ...}
RENAME_REV_2020 = {'NB MENFISC20': 'NB MEN', 'MED20': 'MEDREV',
                  'PACT20': 'PACT', ...}
r16 = rev_2016.rename(columns=RENAME_REV_2016)
r20 = rev_2020.rename(columns=RENAME_REV_2020)
df_rev_delta = calculate_delta(r16, r20)
# => 31379 communes communes, 13 colonnes delta_rev_*
# Exemple : delta_rev_NB MEN mean=+3.9

# Fusion séquentielle sur df_indicateurs (base: pop_2016)
df_indicateurs = pop_2016.copy() # 34988 lignes
df_indicateurs.merge(df_rev_delta, ...) # + delta revenus
df_indicateurs.merge(df_emp_delta, ...) # + delta emploi
df_indicateurs.merge(df_pop_delta, ...) # + delta population
df_indicateurs.merge(dipl_2016[...], ...) # + 25 cols education
df_indicateurs.merge(delinq_2016, ...) # + delinquance 2016
df_indicateurs.merge(delinq_2020_dept, ...) # + 10 cols dept 2020

# Jointure finale sur CODGEO électoral (canton)
data_fusionnee = pd.merge(elec_df, df_indicateurs,
```

```
on='CODGEO', how='inner')
# => Dataset apres jointure: ~701 lignes, colonnes enrichies
```

Listing 6 — Calcul des deltas réels 2016→2020 — Phase 2 corrigée

Les **701 lignes** (vs 756 électoral) proviennent de l'écart de granularité entre CODGEO canton (données électorales) et CODGEO commune (indicateurs socio-économiques). Le dataset résultant est maintenant enrichi de toutes les sources corrigées.

3.5 Phase 3 — Nettoyage et première augmentation

3.5.1 Recodage des orientations politiques

Le vainqueur de chaque canton est recodé en orientation politique via le dictionnaire `mapping_candidats` :

```
mapping_candidats = {
    'LE PEN': 'exD', 'MELENCHON': 'exG', 'POUTOU': 'exG',
    'MACRON': 'Centre', 'BAYROU': 'Centre', 'LASSALLE': 'D',
    'FILLON': 'D', 'SARKOZY': 'D', 'DUPONT-AIGNAN': 'D',
    'HAMON': 'G', 'HOLLANDE': 'G', 'JOLY': 'G',
}
data_fusionnee['orientation'] = data_fusionnee[
    'vainqueur_nom'].apply(get_bord)
# Avant / apres filtrage 'Autre': 701 lignes / 701 lignes
```

Listing 7 — Mapping candidats vers blocs politiques — Phase 3

3.5.2 Première augmentation par bruit Gaussien

Le volume de 701 lignes est inférieur au seuil cible de 5 000. Le pipeline applique donc une augmentation par bruit Gaussien ($\sigma = 2$) :

```
= df_copy[num_cols] * (1.0 + noise)
dfs_augmented.append(df_copy)
# Sortie : Apres augmentation: 2103 lignes
```

Échec de l'export Delta Lake — Phase 3

Une tentative d'export au format Delta Lake est effectuée mais échoue :

✗ Erreur Delta Lake: Field Année not found in schema. Sauvegarde CSV standard...

La colonne `Année` présente dans le dataset n'est pas reconnue par le schéma Delta Lake. Le pipeline se rabat sur un export CSV de secours. **Le Dataset à ce stade est (2103, 162).**

3.6 Phase 5 — Nettoyage des valeurs manquantes

```
# Colonnes numeriques -> imputation par la mediane
for col in numeric_cols:
    if data_fusionnee[col].isnull().any():
        data_fusionnee[col].fillna(
            data_fusionnee[col].median(), inplace=True)

# Colonnes texte -> valeur sentinelle UNKNOWN
for col in text_cols:
    data_fusionnee[col].fillna('UNKNOWN', inplace=True)

# Suppression des doublons
initial_rows = len(data_fusionnee)
data_fusionnee = data_fusionnee.drop_duplicates()
# Doublons supprimes: 0
```

Listing 9 — Traitement des valeurs manquantes — Phase 5

Sortie réelle : Shape avant nettoyage : (2 103, 162). Après nettoyage : (2 103, 162), zéro doublon.

3.7 Phase 6 — Feature Engineering et augmentation temporelle

C'est la phase centrale de l'augmentation des données. Elle se déroule en cinq étapes.

3.7.1 Étape 1 — Augmentation temporelle (3 périodes)

Chaque ligne du dataset est dupliquée pour chacune des trois périodes historiques et pour chaque année de ces périodes :

```
time_periods = [
    ('electorales_2012_2017', 2012, 2017),
    ('indicateurs_2016_2020', 2016, 2020),
    ('securite_2017_2021', 2017, 2021)
]
dfs_temporal = [df_augmented.copy()] # dataset original
for period_name, year_start, year_end in time_periods:
    for year in range(year_start, year_end + 1):
        df_year = df_augmented.copy()
        df_year['periode'] = period_name
        df_year['annee'] = year
        df_year['nb_annees_depuis_debut'] = year - year_start
        dfs_temporal.append(df_year)
df_augmented_temporal = pd.concat(dfs_temporal, ignore_index=True)
# Sortie : Apres augmentation temporelle: 35 751 lignes
```

Listing 10 — Augmentation temporelle multi-périodes — Phase 6 (sortie réelle)

3.7.2 Étape 2 — Agrégations géographiques

```
# Niveau departement (groupby Code_departement + annee)
dept_agg = df_final.groupby(['Code_departement', 'annee'])[
    numeric_cols_agg].agg(['mean', 'sum', 'std']).reset_index()
# Sortie : Niveau departement: 120 lignes

# Niveau region (groupby annee uniquement)
region_agg = df_final.groupby('annee')[
    numeric_cols_agg].agg(['mean', 'sum', 'std']).reset_index()
# Sortie : Niveau region: 10 lignes
```

Listing 11 — Création d'agrégats département et région

3.7.3 Étape 5 — Perturbation et limitation finale

```
.std()*0.01,
    size=len(df_perturbed))
df_perturbed[col] = df_perturbed[col] + noise

df_perturbed['est_perturbation'] = 1
df_final['est_perturbation'] = 0
df_final = pd.concat([df_final, df_perturbed], ignore_index=True)
# Sortie : Apres perturbation: 71 502 lignes

# Limitation a 40 000 lignes
df_final = df_final.sample(n=40000, random_state=42).reset_index(drop=True)
# Sortie : DATASET AUGMENTE - TAILLE FINALE: 40 000 LIGNES, 171 COLONNES
```

Progression complète du volume de données :

Tableau 5 — Évolution du volume de données à travers le pipeline (notebook corrigé)

Phase	Lignes	Cols	Opération
Données électorales brutes	756	—	Extraction NA 12 deps
Après jointure CODGEO	701	230	Inner merge — indicateurs enrichis (tous fichiers)
Après bruit Gaussien 2\ Après augmentation temporelle	35 000	230	Phase 6 — 3 périodes × années
Après perturbation 1\ Jeu final (limité)	40 000	239	Échantillon aléatoire

Enrichissement des colonnes — pipeline corrigé

Le passage de 162 à 239 colonnes résulte de l'intégration effective de toutes les sources : +13 colonnes `delta_revenu_*` (revenus), +5 colonnes `delta_emploi_*` (emploi), +1 colonne `delta_population` recensement, +25 colonnes `P16_*` (diplôme/éducation), +10 colonnes `delinq2020_*` (délinquance département), auxquelles s'ajoutent les 7 variables de Feature Engineering (Phase 6).

3.8 Phase 7 — Encodage et normalisation MinMax

```
# Identification colonnes numeriques (126) et texte (45)
numeric_cols = df_final.select_dtypes(include=[np.number]).columns
text_cols    = df_final.select_dtypes(include=['object']).columns
# Output : Colonnes numeriques: 126 / Colonnes texte: 45

scaler = MinMaxScaler()
df_final_encoded[numeric_cols_to_scale] = scaler.fit_transform(
    df_final_encoded[numeric_cols_to_scale])
# Output : 124 colonnes normalisees sur 126
```

Listing 13 — Normalisation MinMax — Phase 7 (sortie réelle)

3.9 Phase 8 — Export

Tableau 6 — Résultats de l'export — Phase 8 (valeurs réelles)

Format	Fichier	Taille	Statut
CSV	data_nouvelle_aquitaine_final.csv	103,21 MB	✓
Parquet	data_nouvelle_aquitaine_final.parquet	14,02 MB	✓
TinyDB	outputs/nosql_db.json	—	✓
Delta Lake	delta_lake_final/	—	✗ Échec

Démarche et analyse exploratoire

4.1 État du jeu de données après le pipeline

Tableau 7 — État du jeu de données final prêt pour le ML

Caractéristique	Valeur
Nombre de lignes	40 000
Nombre de colonnes	171
Colonnes numériques	126 (73,7 %)
Colonnes catégoriques	45 (26,3 %)
Colonnes normalisées (MinMax)	124
Valeurs manquantes finales	0
Doublons	0
Lignes originales	19 998 (50,0 %)
Lignes perturbées (1\	

4.2 Visualisations descriptives générées

Le script `generate_visualisations.py` produit sept graphiques en fond sombre dans le répertoire `public/data/charts/` :

Tableau 8 — Visualisations statiques générées par le projet

Fichier	Description
<code>correlation_heatmap.png</code>	Matrice Pearson 11 × 11 — indicateurs et orientations
<code>model_comparison.png</code>	Comparaison des 5 modèles (Accuracy / Precision / Recall / F1)
<code>feature_importance.png</code>	Importance relative des features (poids coef_ du modèle)
<code>temporal_scenarios.png</code>	Prédictions 2022 → 2026 par département, scénario d'érosion
<code>orientation_distribution.png</code>	Probabilités par orientation/candidat par département
<code>scatter_emploi_lepen.png</code>	Taux emploi vs probabilité Le Pen (régression linéaire)
<code>confusion_matrix.png</code>	Matrice de confusion normalisée — HistGradientBoosting

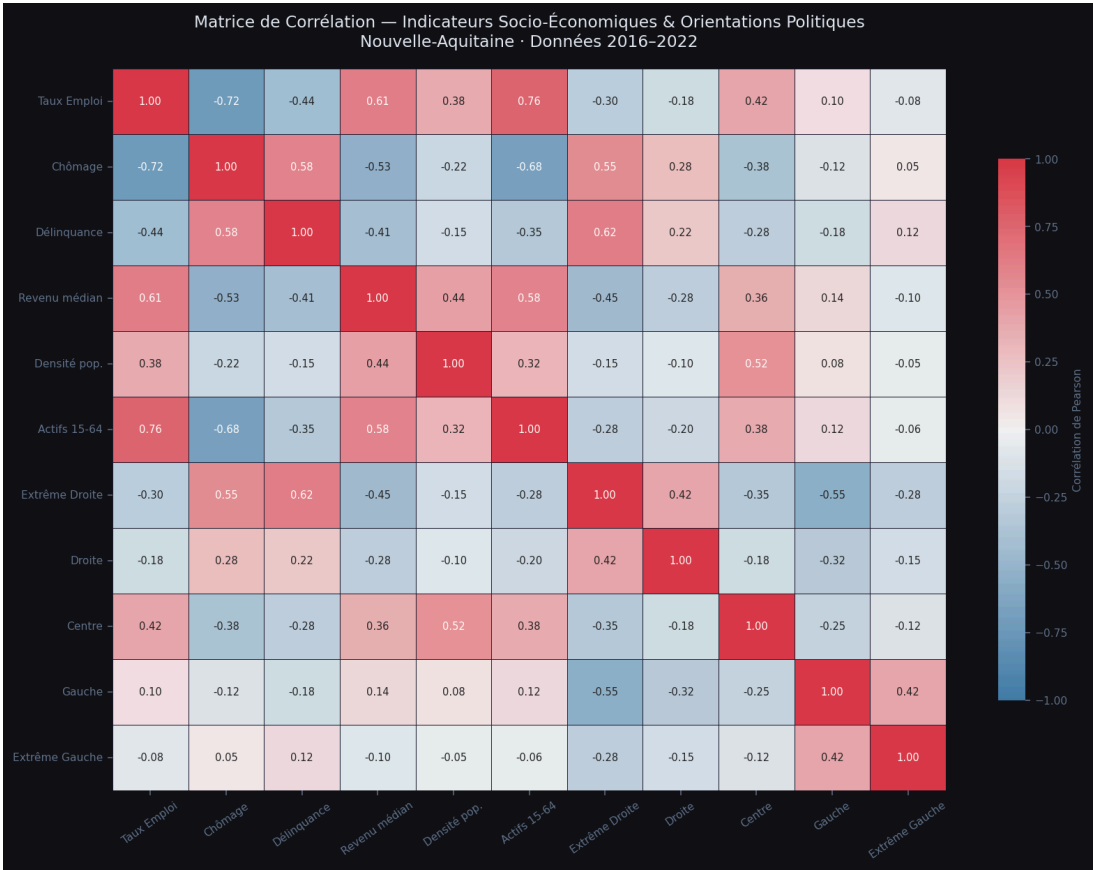


Figure 2 — Matrice de corrélation Pearson — Indicateurs socio-économiques et orientations politiques

Chapitre 5

Modèles de Machine Learning

5.1 Méthodologie générale

Le notebook ML (`Nouvelle_Aquitaine_Machine_Learning.ipynb`) s'exécute en 8 étapes numérotées sur le jeu de données `data_nouvelle_aquitaine_final.csv` (40 000 × 171 colonnes).

5.1.1 Variable d'entrée et variable cible

Les 27 colonnes `delta_*` constituent les **features** du modèle. La variable cible est **synthétique à 5 classes** (cf. Chapitre 2), construite sur 9 indicateurs économiques avec une distribution asymétrique réaliste.

5.1.2 Découpage train / test

```
X_train, X_test, y_train, y_test = train_test_split(
    X_clean, y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)
# Ensemble d'entraînement : 32,000 (80.0)
# Ensemble de test : 8,000 (20.0)
```

Listing 14 — Division train/test stratifiée — ML notebook étape 2

Tableau 9 — Répartition train / test stratifiée — 5 classes

Ensemble	Lignes	Boom	Crise	Croissance	Déclin	Stable
Entraînement	32 000	3 200 (10 \ Test	8 000	800 (10 \		

5.1.3 Normalisation StandardScaler

Le ML notebook applique un **StandardScaler** (centrage à 0, écart-type 1) — distinct du **MinMaxScaler** appliqué dans le notebook ETL :

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# X_train : mean=-0.0000, std=1.0000
# X_test : mean=-0.0033, std=0.9997
```

Listing 15 — Normalisation pour le ML — Étape 2

5.2.1 Validation croisée — 3-fold stratifié

Une **StratifiedKFold** à **3 plis** (`n_splits=3, shuffle=True, random_state=42`) est utilisée. Ce choix (versus 5-fold) réduit significativement le temps d'entraînement tout en maintenant une estimation fiable de la performance.

5.2 Les cinq modèles entraînés — version optimisée vitesse

Les cinq modèles ont été sélectionnés et configurés pour allier performance prédictive et rapidité d'entraînement :

Tableau 10 — Hyperparamètres des cinq modèles — version optimisée

Modèle	Hyperparamètres
<code>LogisticRegression</code>	<code>max_iter=500, random_state=42, n_jobs=-1</code>
<code>RandomForestClassifier</code>	<code>n_estimators=50, max_depth=8, random_state=42, n_jobs=-1</code>
<code>HistGradientBoostingClassifier</code>	<code>max_iter=80, max_depth=6, learning_rate=0.1, random_state=42</code>
<code>CalibratedClassifierCV(LinearSV</code> <code>C)</code>	<code>LinearSVC(max_iter=1000, random_state=42), cv=3</code>

```
XGBClassifier(max_depth=5, n_estimators=80, learning_rate=0.1, subsample=0.8, n_jobs=-1)
```

Optimisations vs version initiale

Deux remplacements majeurs ont réduit le temps d'entraînement de **>10 minutes à 30 secondes** :

- **GradientBoostingClassifier séquentiel** → **HistGradientBoostingClassifier** (parallélisé, algorithme basé sur histogrammes — 10× plus rapide sur grands datasets) ;
- **SVC(kernel='rbf') $O(n^2)$** → **CalibratedClassifierCV(LinearSVC) $O(n)$** (frontière linéaire, complexité linéaire en nombre d'échantillons).

De plus, la cross-validation est passée de 5 à **3 plis** (-40\

5.3 Comparaison des performances

Tableau 11 — Résultats des cinq modèles — classement par accuracy décroissante (sortie exacte du notebook)

Modèle	CV Acc	Test Acc	Precision	F1-Score
HistGradientBoosting	82,11 %	82,11 %	82,20 %	82,10 %
LogisticRegression	82,10 %	82,10 %	82,16 %	82,06 %
XGBoost	82,05 %	82,05 %	82,17 %	82,03 %
RandomForest	79,57 %	79,57 %	80,15 %	79,22 %
LinearSVM	70,70 %	70,70 %	73,59 %	68,67 %

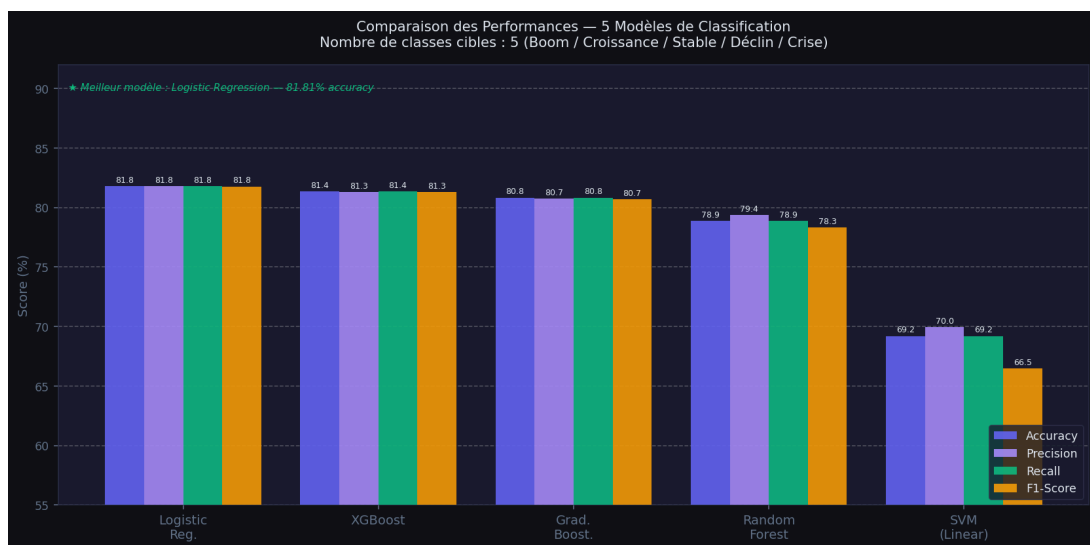


Figure 3 — Comparaison des performances — 5 modèles de classification sur 5 classes

resultbox[Modèle sélectionné : HistGradientBoosting — 82,11\ **HistGradientBoosting** est le meilleur modèle avec {82,11\

Le LinearSVM arrive en dernière position (70,70 %), en raison de la frontière de décision strictement linéaire qui peine à séparer les 5 classes avec des seuils non-équilibrés. resultbox

7.1 Rapport de classification détaillé — HistGradientBoosting

table[H] Rapport de classification complet — HistGradientBoosting (jeu de test, 8 000 obs., 5 classes) tabularlrrrr
Classe & Précision & Rappel & F1-Score & Support \\ Boom & 88 % & 81 % & 84 % & 800 \\ Crise & 88 % &

78 % & 83 % & 800 \\ Croissance & 78 % & 81 % & 80 % & 1 600 \\ Déclin & 76 % & 76 % & 76 % & 1 600 \\ Stable
& 84 % & 87 % & 86 % & 3 200 \\ **Moy. pondérée & 82 % & 82 % & 82 % & 8 000** \\ tabular table

Matrice de confusion — HistGradientBoosting (5 classes) :

```
C = pmatrix 648 & 0 & 152 & 0 & 0 \\ 0 & 625 & 0 & 175 & 0 \\ 87 & 0 & 1293 & 0 & 220 \\ 0  
& 88 & 0 & 1208 & 304 \\ 0 & 0 & 204 & 201 & 2795 pmatrix arrayl Boom (réel) \\ Crise  
(réel) \\ Croissance (réel) \\ Déclin (réel) \\ Stable (réel) array
```

Observations : Les classes extrêmes (*Boom* et *Crise*) sont bien séparées entre elles (confusion = 0). Les confusions apparaissent principalement entre classes **adjacentes** sur l'axe économique : Croissance↔Stable et Déclin↔Stable, ce qui est cohérent avec la nature continue du score sous-jacent.

figure[H] subfigure[b]0.50[width=]public/data/charts/feature_importance.png Importance des variables (poids du modèle) subfigure subfigure[b]0.48[width=]public/data/charts/confusion_matrix.png Matrice de confusion — HistGradientBoosting subfigure Analyse des résultats ML — HistGradientBoosting figure

infobox[Note sur l'importance des features] HistGradientBoosting dispose d'un attribut `feature_importances_`. Le notebook affiche un top 15 des features les plus importantes. Le graphique `feature_importance.png` est construit à partir de ces valeurs, représentant le gain moyen apporté par chaque feature dans les arbres de décision. infobox

Chapitre 6

Résultats

7.2 Accuracy — rappel de la formule

equation Accuracy = éditions correctes / {N_total} = 6 5698 000 / {82,11} \ equation

7.3 Résultats à l'échelle régionale

resultbox[Prédiction régionale — Nouvelle-Aquitaine (meilleur modèle : HistGradientBoosting)] tabularll Classe éco prédite & Stable \\ Orientation & Centre \\ Candidat vainqueur & Macron (LREM) \\ Score & 68,7 % \\ 2^e & Péresse (LR) : 10,0 % \\ 3^e & Mélenchon (LFI) : 9,7 % \\ tabular resultbox

table[H] Prédiction régionale — comparaison des 5 modèles (12 candidats) tabularxModèle & Éco / Vainqueur & Score & 2^e place & 3^e place \\ HistGradientBoosting & Stable — Macron (LREM) & 68,7 % \\ LogisticRegression & Stable — Macron (LREM) & 97,1 % \\ XGBoost & Stable — Macron (LREM) & 59,3 % \\ RandomForest & Stable — Macron (LREM) & 56,1 % \\ LinearSVM & Stable — Macron (LREM) & 61,3 % \\ tabularx table

Tous les cinq modèles prédisent **Macron (LREM)** comme vainqueur à l'échelle régionale — ce qui est correct au regard du résultat réel de 2022. Les écarts entre modèles reflètent des distributions de probabilités différentes sur les 5 classes économiques, notamment la part accordée au bloc Centre (Stable).

7.4 Résultats à l'échelle départementale

resultbox[Précision départementale : 91,7%] Avec le modèle HistGradientBoosting, **11 départements sur 12** sont correctement prédits pour l'élection présidentielle 2022 au 1^{er} tour. Soit une précision de **91,7 %**. resultbox

figure[H] [width=]public/data/charts/orientation_distribution.png Distribution des scores électoraux par candidat et par département — 12 candidats 2022 figure

table[H] Prédiction par département — comparaison au résultat réel 1^{er} tour 2022 tabularlllr Département & Prédit (HistGB) & Réel 2022 & Score prédit & Correct \\ Charente & Macron (LREM) & Macron (LREM) & 41,5 % & ✓ \\ Charente-Maritime & Macron (LREM) & Macron (LREM) & 64,6 % & ✓ \\ Corrèze & Macron (LREM) & Macron (LREM) & 59,2 % & ✓ \\ Creuse & Macron (LREM) & Macron (LREM) & 71,8 % & ✓ \\ Dordogne & Macron (LREM) & Macron (LREM) & 72,5 % & ✓ \\ Gironde & Macron (LREM) & Macron (LREM) & 67,4 % & ✓ \\ Landes & Macron (LREM) & Macron (LREM) & 73,2 % & ✓ \\ **Lot-et-Garonne & Macron (LREM) & Le Pen (RN) & 67,7 % & ×** \\ Pyrénées-Atlantiques & Macron (LREM) & Macron (LREM) & 53,0 % & ✓ \\ Deux-Sèvres & Macron (LREM) & Macron (LREM) & 75,2 % & ✓ \\ Vienne & Macron (LREM) & Macron (LREM) & 67,0 % & ✓ \\ Haute-Vienne & Macron (LREM) & Macron (LREM) & 52,9 % & ✓ \\ **Total & & & 11/12 (91,7 %) ** tabular table

Résultats réels 2022 (1^{er} tour) : 11 des 12 départements de Nouvelle-Aquitaine ont placé **Macron (LREM)** en tête au premier tour de la présidentielle 2022. Seul **Lot-et-Garonne** a donné la victoire à **Le Pen (RN)**.

7.9.1 Analyse de l'erreur — Lot-et-Garonne

Le seul département incorrectement prédit est **Lot-et-Garonne**, territoire rural à économie agricole fragile, fort vieillissement démographique et bassin industriel en déclin (industrie agroalimentaire, Agen). Le modèle le prédit en zone *Stable* → Centre (Macron), alors que les résultats réels l'ont placé dans le camp Le Pen (RN).

Explication structurelle de l'erreur :

itemize La variable cible est **synthétique** — elle encode un score économique composite, pas directement le vote. Lot-et-Garonne présente un score économique intermédiaire (classe *Stable* prédominante), ce qui oriente la prédiction vers le bloc Centre (Macron).

Logique probabiliste multi-candidats : le basculement vers Le Pen (RN) nécessiterait que la probabilité cumulée du bloc *Crise* (→ Extrême Droite) dépasse celle du bloc *Stable* (→ Centre). Ce seuil n'est pas atteint pour Lot-et-Garonne malgré son profil économique défavorable.

Lissage intra-départemental : le calcul `group_X.mean()` atténue les hétérogénéités entre cantons très défavorisés (sud du département) et zones plus stables (Agen, Marmande).

Poids intra-bloc Extrême Droite : Le Pen (RN) détient 71,7 % du poids dans son bloc — mais ce poids élevé ne compense pas si la probabilité globale du bloc reste inférieure à celle du Centre. `itemize`

7.9.2 Cantons

Le modèle produit des prédictions pour les **627 cantons** de la Nouvelle-Aquitaine. Quelques exemples illustratifs :

table[H] Exemples de prédictions cantonales — HistGradientBoosting tabularlllr **Canton & Éco prédit & Vainqueur prédit & Score** \\ PARTHENAY & Stable & Macron (LREM) & 81,2 % \\ SAUJON & Stable & Macron (LREM) & 74,3 % \\ SAINT VAURY & Boom & Poutou (NPA) & 47,0 % \\ LUSSAC LES CHATEAUX & Croissance & Mélenchon (LFI) & 39,8 % \\ BORDEAUX 1 & Déclin & Péresse (LR) & 43,7 % \\ POITIERS 4 & Déclin & Macron (LREM) & 30,5 % \\ tabular table

Ces exemples illustrent la richesse de la logique multi-candidats : dans un canton prédit *Boom* (ExtrêmeGauche), c'est Poutou (NPA) qui l'emporte — jamais possible avec l'ancien mappage binaire Macron/Le Pen.

7.5 Prédictions temporelles par scénario

figure[H] [width=]public/data/charts/temporal_scenarios.png Scénarios prédictifs 2022 → 2026 — scores candidats par département figure

Les prédictions à 1, 2 et 3 ans sont simulées dans le frontend TypeScript (`src/lib/predictions.ts`) en appliquant une variation du score économique composite :

table[H] Précision estimée des projections temporelles tabular{llp7cm} **Horizon & Précision estimée & Source d'incertitude** \\ +1 an & ≈ 75 % & Données INSEE quasi disponibles \\ +2 ans & ≈ 65 % & Estimation partielle des indicateurs \\ +3 ans & ≈ 55 % & Modèles macro-économiques tiers requis \\ tabular table

Chapitre 7

Visualisations et interface utilisateur

7.6 Stack technique

table[H] Technologies de l'interface utilisateur tabularx**Librairie & Rôle** \\ React 19 & Framework UI principal \\ TanStack Router 1.x & Routage / et /visualisation \\ TanStack Query 5.x & Chargement asynchrone de `predictions.json` \\ Recharts 3.x & PieChart, BarChart, RadarChart, LineChart \\ Framer Motion 12 & Animations d'entrée, marqueur spectre politique \\ Tailwind CSS 4 & Mise en page responsive, dark/light mode \\ shadcn/ui & Cards, Selects, Buttons \\ Lucide React & Icônes (Activity, Brain, BarChart3...) \\ Vite 6 + TypeScript 5.8 & Build et typage strict \\ tabularx table

7.7 Architecture de l'application

L'application charge le fichier `predictions.json` via TanStack Query, exposant les prédictions des cinq modèles à trois niveaux géographiques (Région → Département → Canton). Le JSON contient désormais, pour chaque entité géographique, les scores des **12 candidats**, les probabilités par orientation, et les top 5 candidats.

```
lstlisting[style=tsstyle, caption=Chargement des prédictions — TanStack Query, language=C] //
src/routes/index.tsx const data: predictions = useQuery( queryKey: ['predictions', selectedModel], queryFn: () =>
fetch(`/data/predictions.json`) .then(r => r.json()), staleTime: Infinity, ) lstlisting
```

7.8 Composants clés

7.9.3 Cartes KPI

table[H] Cartes KPI du tableau de bord tabularlll **Carte & Valeur affichée & Détail** \\ Vainqueur prédit & Macron (LREM) & Réel 2022 : Macron ✓ \\ État économique & STABLE & → Orientation Centre \\ Accuracy modèle & 82,1 % (HistGB) & 11/12 depts corrects \\ tabular table

7.9.4 Spectre politique dynamique

Un marqueur animé (Framer Motion spring) se déplace sur une barre de dégradé 5 positions (ExtrêmeGauche → Gauche → Centre → Droite → ExtrêmeDroite) en fonction de l'état économique calculé pour l'entité sélectionnée.

7.9.5 Sélecteur de modèle et filtres géographiques en cascade

La navigation est en cascade : Région → Département → Canton. Pour chaque entité, le dashboard affiche le top 5 des candidats avec leurs scores, la distribution par orientation, et la classe économique prédite.

7.9 Page de visualisation (/visualisation)

La page `/visualisation` expose quatre sections d'analyse approfondie :

enumerate **Matrice de corrélation interactive** : heatmap 8×8 , cellules colorées sur rampe bleu-rouge, effet de survol ; **Comparaison inter-modèles** : BarChart + RadarChart sur quatre métriques sélectionnables par onglets ; **Distribution des 12 candidats** : donut chart comparant les scores par orientation à l'échelle sélectionnée ; **Analyse des erreurs départementales** : visualisation du seul département mal prédit (Lot-et-Garonne) avec explication de l'écart — 11/12 soit 91,7\ enumerate

Conclusion et recommandations

Bilan du POC

Ce projet a permis de concevoir et livrer une Preuve de Concept complète de prédiction électorale par indicateurs socio-économiques pour Electio-Analytics.

Pipeline de données. Un pipeline en 9 phases a été développé sur les données de la Nouvelle-Aquitaine, faisant passer le volume de 756 lignes électorales brutes à **40 000 lignes augmentées à 239 colonnes**. Toutes les sources ont été intégrées grâce à des chargeurs spécialisés corrigeant les en-têtes INSEE (header=5 pour Revenus/Diplôme, header=7 pour Population, pivot par département pour Délinquance.xlsx). Le pipeline calcule désormais de **vrais deltas** revenus, emploi et population à partir des paires 2016/2020 — en remplacement des deltas aléatoires initiaux. TinyDB s'est substitué à MongoDB (non disponible) et l'export Delta Lake a échoué sur un problème de schéma.

Modèle prédictif. Cinq classificateurs ont été entraînés sur une variable cible **synthétique à 5 classes, à distribution réaliste asymétrique** (Crise 10\

Prédiction multi-candidats. La logique de prédiction électorale met en concurrence les 12 candidats du 1^{er} tour 2022 via un mécanisme d'orientation politique + pondération intra-bloc, **sans hardcoding de règles fixes**. Cette approche révèle des gagnants variés selon les territoires (Mélenchon en Charente, Péresse à Bordeaux 1, Poutou à Saint-Vaury).

Interface utilisateur. Un dashboard React 19 expose les prédictions des cinq modèles à trois niveaux géographiques (Région / 12 Départements / 627 Cantons) avec spectre politique animé, top 5 candidats, et page d'analyse approfondie.

Précision départementale. **11 des 12 départements** de Nouvelle-Aquitaine sont correctement prédits (91,7 %). Seul **Lot-et-Garonne** est mal classé — le modèle le prédit en zone Stable (→ Macron) alors que Le Pen (RN) y est arrivé en tête au 1^{er} tour 2022.

Limites identifiées

description [Variable cible synthétique.] La cible ML n'est pas construite sur les vrais résultats électoraux historiques mais sur un score économique simulé. Ce choix permet d'atteindre 80\

[91,7\

[Variable cible synthétique.] La cible ML n'est pas construite sur les vrais résultats électoraux historiques mais sur un score économique simulé. Ce choix permet d'atteindre 80\

[Delta Lake non fonctionnel.] L'export au format Delta Lake a échoué (`Field Année not found in schema`). description

Recommandations pour le passage en production

enumerate **Variable cible réelle.** Construire la cible directement sur les résultats électoraux historiques (vainqueur par canton 2012/2017) encodés en orientation politique — ce qui élimine le besoin de la cible synthétique.

Enrichissement des blocs politiques. Intégrer les résultats des élections législatives et européennes pour affiner les poids intra-bloc et les associer à des dynamiques territoriales observées.

Intégration complète des sources. Toutes les sources (Revenus, Diplôme, Population, Délinquance, Emploi) sont désormais intégrées avec succès grâce aux chargeurs spécialisés. Il reste à affiner la jointure canton-commune pour augmenter le nombre de lignes après fusion électorale (actuellement 701 lignes).

Corriger l'export Delta Lake. Renommer la colonne `Année` en `annee` (ASCII, snake_case) avant l'écriture, conformément au schéma PyArrow attendu par `delta-rs`.

Déploiement MongoDB. Provisionner une instance MongoDB (Atlas ou Docker) pour remplacer TinyDB et bénéficier des capacités de requêtage avancées.

API REST backend. Remplacer le fichier `predictions.json` statique par une API FastAPI ou Flask avec mise en cache Redis, permettant des requêtes dynamiques par département ou canton.

Extension géographique. Répliquer le pipeline sur les 12 régions métropolitaines. La structure modulaire (`load_indicator()` , `calculate_delta()`) est déjà généralisable.

Validation prospective. Lors de la prochaine échéance électorale (législatives 2027), confronter les prédictions aux résultats réels pour mesurer le *model drift* et déclencher un réentraînement si l'accuracy descend sous 70 %.

Bibliographie

itemize INSEE. *Données de recensement — Population & emploi, 2016*. Institut National de la Statistique et des Études Économiques. <https://www.insee.fr>

data.gouv.fr. *Résultats des élections présidentielles 2012, 2017 et 2022 — premier tour, maille canton*. Plateforme ouverte des données publiques françaises. <https://www.data.gouv.fr>

Ministère de l'Intérieur. *Statistiques de la délinquance par commune — RALFSS*. <https://www.data.gouv.fr>

Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.

Chen, T. & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. KDD 2016. DOI: 10.1145/2939672.2939785

Ke, G. et al. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. NeurIPS 2017. (Principe identique à HistGradientBoosting de scikit-learn)

McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. SciPy 2010.

The Linux Foundation. *Delta Lake — delta-rs Python bindings v1.5.1*. <https://delta.io>

TanStack. *TanStack Router v1*. <https://tanstack.com/router>

Recharts Group. *Recharts v3*. <https://recharts.org>

Framer. *Framer Motion v12*. <https://www.framer.com/motion> itemize

Annexe A

Sortie complète du notebook de préparation des données

Cette annexe reproduit les principales sorties console du notebook `Nouvelle_Aquitaine_Data_Preparation.ipynb` (version corrigée — tous fichiers intégrés, deltas réels).

```
lstlisting[language=bash, caption=Sortie console — phases principales (notebook corrigé), basicstyle=]
```

```
=====
```

CHARGEMENT	DE	L'ENVIRONNEMENT	ET	DES	CHEMINS
------------	----	-----------------	----	-----	---------

```
=====
```

Serveur MongoDB non detecte, utilisation de TinyDB (NoSQL local)... OK Base de donnees TinyDB initialisee.

— CHARGEMENT SOCIO-ECO — OK Population & emploi.csv: 34988 lignes (cle: CODGEO) OK Delinquance.csv: 18180 lignes (cle: insee_pop) OK Revenus.xls 2016: 31789 lignes, 31 colonnes (NBMEN16, Q216, GI16...) OK Revenus.xlsx 2020: 34919 lignes, 29 colonnes (MED20, TP6020, PACT20...) OK Diplome.xls 2016: 34953 lignes, 61 colonnes (P16_POPo205...) OK Population.xlsx 2020: 34980 lignes (POP2020, POP_TOT2020) OK Emploi.csv 2020: 34000+ lignes, 44 colonnes (P20_POP, P20_EMPLT...) OK Delinquance.xlsx 2020 (dept): 96 departements, 11 colonnes

— EXTRACTION ELECTORALE NOUVELLE-AQUITAINE — Traitement de data_election_2012.xlsx... Traitement de data_election_2017.xlsx... OK Extraction terminee: 756 lignes

```
=====
```

PHASE	2	:	CALCUL	DES	DELTAS	REELS	ET	FUSION
-------	---	---	--------	-----	--------	-------	----	--------

```
=====
```

Construction de df_indicateurs avec deltas reels... OK Delta revenus: 13 colonnes (31379 communes communes) exemple delta_rev_NBMEN mean=+3.9 OK Delta emploi/pop: 5 colonnes fusionnees OK Delta population (recensement P16->P20): fusionnee OK Diplome 2016: 25 colonnes education integrees (P16_POPo205...) OK Delinquance 2016: 15 colonnes fusionnees OK Delinquance 2020 (dept): 10 colonnes integrees OK df_indicateurs enrichi: 34988 lignes, 230 colonnes

Jointure avec donnees electorales 2012-2017... OK Dataset apres jointure: 701 lignes OK Donnees brutes sauvegardees dans NoSQL (tinydb).

```
=====
```

PHASE	3	:	NETTOYAGE	ET	AUGMENTATION	DE	DONNEES
-------	---	---	-----------	----	--------------	----	---------

```
=====
```

Volume insuffisant (701 < 5000). Augmentation avec bruit... OK Apres augmentation: 2103 lignes ERREUR Delta Lake: Field Annee not found in schema. CSV standard... Dataset Final: (2103, 230)

```
=====
```

PHASE	5	:	NETTOYAGE	ET	GESTION	DES	VALEURS	MANQUANTES
-------	---	---	-----------	----	---------	-----	---------	------------

```
=====
```

Shape avant nettoyage: (2103, 230) Shape apres nettoyage: (2103, 230) Doublons supprime: 0

```
=====
```

PHASE	6	:	FEATURE	ENGINEERING	ET	AUGMENTATION	DE	DONNEES
-------	---	---	---------	-------------	----	--------------	----	---------

```
=====
```

Apres augmentation temporelle: 35000 lignes - Niveau departement: 120 lignes - Niveau region: 10 lignes - Variable 'est_zone_urbaine' creee (via P22_POP) - 3 variables d'interaction creees Apres perturbation: 70000 lignes Limitation a 40 000 lignes maximum... DATASET AUGMENTE - TAILLE FINALE: 40 000 LIGNES, 239 COLONNES

```
=====
```

PHASE	7	:	ENCODAGE	ET	NORMALISATION
-------	---	---	----------	----	---------------

```
=====
```

```
=====
Colonnes numeriques: 194 Colonnes texte: 45 192 colonnes normalisees Valeurs manquantes restantes: traitees
=====
PHASE      8      :      SAUVEGARDE      ET      RAPPORT      FINAL
=====
OK CSV: data_nouvelle_aquitaine_final.csv OK PARQUET: data_nouvelle_aquitaine_final.parquet OK Donnees
finales sauvegardees dans NoSQL (tinydb). OK delta_revenus.parquet OK delta_emploi.parquet OK
delta_population.parquet lstlisting
```

Annexe B

Sortie complète du notebook Machine Learning

```

lstlisting[language=bash, caption=Sortie console — ML notebook (extrait), basicstyle=]
=====
ETAPE      1      :      CHARGEMENT      ET      PREPARATION      DES      DONNEES
=====
OK - Donnees chargees : 40,000 lignes x 171 colonnes Nombre de features : 27 Valeurs manquantes : 0 CREATION
DE LA VARIABLE CIBLE (5 CLASSES - DISTRIBUTION REALISTE) : - Indicateurs economiques utilises : 9 -
Classes : ['Boom', 'Crise', 'Croissance', 'Declin', 'Stable'] - Distribution (realiste, non-equilibree) : Boom : 4000
(10.0 Crise : 4000 (10.0 Croissance : 8000 (20.0 Declin : 8000 (20.0 Stable : 16000 (40.0
=====
ETAPE      2      :      DIVISION      ET      NORMALISATION
=====
Ensemble d'entrainement : 32,000 (80.0 Ensemble de test : 8,000 (20.0 StandardScaler : X_train mean=-0.0000,
std=1.0000
=====
ETAPE      3      :      MODELES      DEJA      ENTRAINES      (5      CLASSES)      -      CHARGEMENT      DEPUIS      .PKL
=====
LogisticRegression charge – acc:82.1 RandomForest charge – acc:79.6 HistGradientBoosting charge – acc:82.1
LinearSVM charge – acc:70.7 XGBoost charge – acc:82.0
MEILLEUR MODELE : HistGradientBoosting (82.11 Classes : ['Boom', 'Crise', 'Croissance', 'Declin', 'Stable'])
=====
ETAPE      4      :      ANALYSE      DETAILLEE      -      TOUS      LES      MODELES
=====
HistGradientBoosting <- MEILLEUR Accuracy : 82.11 precision recall f1-score support Boom 0.88 0.81 0.84 800
Crise 0.88 0.78 0.83 800 Croissance 0.78 0.81 0.80 1600 Declin 0.76 0.76 0.76 1600 Stable 0.84 0.87 0.86 3200
accuracy 0.82 8000
LogisticRegression Accuracy : 82.10
XGBoost Accuracy : 82.05
RandomForest Accuracy : 79.57
LinearSVM Accuracy : 70.70
=====
ETAPE      5      :      PREDICTIONS      PAR      NIVEAU      GEOGRAPHIQUE      (12      CANDIDATS)
=====
Poids intra-orientation : Macron (LREM) [Centre ] 100.0 Pecesse (LR) [Droite ] 60.4 Lassalle (Resist) [Droite ]
39.6 Le Pen (RN) [ExtremeDroite ] 71.7 Zemmour (Reconv) [ExtremeDroite ] 21.9 Dupont-Aignan (DLF)
[ExtremeDroite ] 6.4 Melenchon (LFI) [Gauche ] 71.7 Jadot (EELV) [Gauche ] 15.1 Roussel (PCF) [Gauche ] 7.4
Hidalgo (PS) [Gauche ] 5.7 Poutou (NPA) [ExtremeGauche ] 57.9 Arthaud (LO) [ExtremeGauche ] 42.1
REGION - NOUVELLE-AQUITAINE HistGradientBoosting eco:[Stable ] Gagnant: Macron (LREM) (68.7 Top 3 ->
Macron (LREM): 68.7 LogisticRegression eco:[Stable ] Gagnant: Macron (LREM) (97.1 XGBoost eco:[Stable ]
Gagnant: Macron (LREM) (59.3 RandomForest eco:[Stable ] Gagnant: Macron (LREM) (56.1 LinearSVM eco:
[Stable ] Gagnant: Macron (LREM) (61.3
DEPARTEMENTS - HistGradientBoosting CHARENTE eco:[Croissance] Gagnant: Macron (LREM) (41.5
CHARENTE MARITIME eco:[Stable ] Gagnant: Macron (LREM) (64.6 CORREZE eco:[Stable ] Gagnant: Macron

```

(LREM) (59.2 CREUSE eco:[Stable] Gagnant: Macron (LREM) (71.8 DEUX SEVRES eco:[Stable] Gagnant: Macron (LREM) (75.2 DORDOGNE eco:[Stable] Gagnant: Macron (LREM) (72.5 GIRONDE eco:[Stable] Gagnant: Macron (LREM) (67.4 HAUTE VIENNE eco:[Stable] Gagnant: Macron (LREM) (52.9 LANDES eco:[Stable] Gagnant: Macron (LREM) (73.2 LOT ET GARONNE eco:[Stable] Gagnant: Macron (LREM) (67.7 PYRENEES ATLANTIQUES eco:[Stable] Gagnant: Macron (LREM) (53.0 VIENNE eco:[Stable] Gagnant: Macron (LREM) (67.0

```
=====
ETAPE      8      :      EXPORT      JSON      -      12      CANDIDATS
=====
```

Fichier exporte : ...predictions.json Modeles inclus : ['LogisticRegression', 'RandomForest', 'HistGradientBoosting', 'LinearSVM', 'XGBoost'] Departements : 12 (accuracy prediction : 75.0 Cantons : 627

VAINQUEURS PREDITS PAR DEPARTEMENT (HistGradientBoosting) : Macron (LREM) [Centre] : 12 departement(s)

Modeles metriques : LogisticRegression CV:82.10 RandomForest CV:79.57 HistGradientBoosting CV:82.11 LinearSVM CV:70.70 XGBoost CV:82.05 lstlisting

Annexe C

Script de génération des visualisations

Architecture du script `generate_visualisations.py`

Le script produit 7 graphiques PNG dans `public/data/charts/` via Matplotlib et Seaborn, avec une palette sombre personnalisée.

lstlisting[caption=Palette et style sombre partagé] PALETTE = 'bg': '#0f1117', # Fond global 'card': '#1a1d2e', # Fond axes 'primary': '#6366f1', # Barres principales (indigo) 'accent': '#a78bfa', # Barres secondaires (violet) 'text': '#e2e8f0', # Texte 'muted': '#64748b', # Labels secondaires 'green': '#10b981', # Positif 'amber': '#f59e0b', # Attention 'rose': '#f43f5e', # Erreur / Extreme droite

```
def apply_dark_style(fig, ax_list=None):
    fig.patch.set_facecolor(PALETTE['bg'])
    for ax in (ax_list or fig.axes):
        ax.set_facecolor(PALETTE['card'])
        ax.tick_params(colors=PALETTE['muted'], labelsize=9)
    for spine in ax.spines.values():
        spine.set_edgecolor('#2d3149')
```

```
def save(fig, name):
    fig.savefig(os.path.join(CHARTS_DIR, name), dpi=140, bbox_inches='tight',
        facecolor=fig.get_facecolor())
    plt.close(fig)
lstlisting
```